Universidade Federal de Uberlândia
Disciplina: Mineração de Dados – Profa. Elaine Ribeiro Faria

Relatório – Trabalho Final: Classificação

Previsão da intenção de compra de visitantes de uma loja on-line

Nome e número de matrícula dos estudantes:

Cassio Lima Silva	12311BCC011
Diogo Vieira de Castro	12311BCC002
Rodrigo Otávio Fernandes Rodrigues	12311BCC008

I Introdução e objetivos

O comércio eletrônico recebe um grande volume de visitas, mas apenas uma pequena parte delas termina em compra. Identificar, a partir do comportamento de navegação, quais sessões têm maior chance de gerar uma compra permite que a loja aja enquanto o visitante ainda está no site: oferecer um cupom, exibir uma recomendação, acionar um atendimento ou simplesmente não gastar recursos de marketing com quem já vai comprar. Por isso, prever a intenção de compra é um problema de classificação com aplicação direta no negócio.

Neste trabalho, o problema investigado é um problema de classificação binária supervisionada: dado o registro de uma sessão de navegação (páginas visitadas, tempo em cada tipo de página, métricas do Google Analytics, data e perfil técnico do visitante), prever se a sessão terminou em compra (`Revenue` = True) ou não (`Revenue` = False). Um aspecto central do problema é o desbalanceamento: apenas 15,5% das sessões terminam em compra, e é justamente essa classe minoritária que interessa identificar.

Objetivos. Aplicar, avaliar e comparar quatro algoritmos de classificação supervisionada (KNN, Random Forest, Gradient Boosting e rede neural MLP) na base Online Shoppers Purchasing Intention. Para isso, o trabalho:

- explora a base para entender seus atributos, a distribuição das classes e seus problemas de qualidade;
- define um pré-processamento comum a todos os algoritmos e outro específico para cada um, justificado pela análise exploratória e pelos requisitos de cada algoritmo;
- testa diferentes parametrizações de cada algoritmo com uma estratégia de avaliação que permita uma comparação justa entre eles;
- escolhe medidas de avaliação adequadas ao desbalanceamento e, com base nelas, interpreta os resultados e recomenda o modelo mais adequado ao problema.

II Descrição da base de dados

Fonte: UCI Machine Learning Repository – `https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset`.

A base tem 12.330 instâncias e 17 atributos preditores mais a classe (18 atributos). Dos 17 preditores, 10 são numéricos (contagens, durações e taxas) e 7 são categóricos (`Month`, `OperatingSystems`, `Browser`, `Region`, `TrafficType`, `VisitorType` e `Weekend`).

Descrição dos atributos

Navegação na sessão

1. **Administrative** (inteiro, 0 a 27): quantidade de páginas administrativas visitadas na sessão, como login, cadastro, conta do usuário e dados pessoais.
2. **Administrative_Duration** (decimal, 0 a 3.398,75 s): tempo total, em segundos, que o visitante passou nas páginas administrativas durante a sessão.
3. **Informational** (inteiro, 0 a 24): quantidade de páginas informativas visitadas, como sobre a empresa, contato, políticas e formas de entrega.
4. **Informational_Duration** (decimal, 0 a 2.549,38 s): tempo total, em segundos, gasto nas páginas informativas.
5. **ProductRelated** (inteiro, 0 a 705): quantidade de páginas de produtos visitadas, como catálogo, detalhes do produto e categorias.
6. **ProductRelated_Duration** (decimal, 0 a 63.973,52 s): tempo total, em segundos, gasto nas páginas de produtos. Em média são cerca de 1.195 s por sessão, bem mais que nas outras categorias.
7. **BounceRates** (decimal, 0 a 0,20): taxa de rejeição média das páginas visitadas. É a porcentagem de visitantes que entram no site por uma página e saem sem fazer nenhuma outra interação registrada pelo servidor de análise.
8. **ExitRates** (decimal, 0 a 0,20): taxa de saída média das páginas visitadas. Para uma página, é a porcentagem de todas as suas visualizações que foram a última da sessão, ou seja, o ponto em que o usuário deixou o site.
9. **PageValues** (decimal, 0 a 361,76): valor médio das páginas que o usuário visitou antes de concluir uma compra. Mostra quanto cada página contribui para gerar receita. A média é de cerca de 5,89, e o valor é 0 em muitas sessões.

Contexto temporal

10. **SpecialDay** (0; 0,2; 0,4; 0,6; 0,8; 1,0): mede o quanto a visita está próxima de uma data comemorativa (Dia das Mães, Dia dos Namorados etc.), quando é mais provável que a sessão termine em compra. O cálculo considera o prazo entre o pedido e a entrega. Por exemplo, no Valentine's Day o valor é diferente de zero entre 2 e 12 de fevereiro, chega ao máximo de 1 no dia 8 e fica em 0 fora desse período, a não ser que esteja perto de outra data especial.
11. **Month** (texto): mês da visita. A base tem 10 meses: Feb, Mar, May, June, Jul, Aug, Sep, Oct, Nov e Dec. Janeiro e abril não aparecem.
12. **Weekend** (booleano, True/False): indica se a visita foi em um fim de semana.

Perfil técnico e de origem do visitante

13. **OperatingSystems** (categórico, códigos 1 a 8): sistema operacional do visitante. Os números são rótulos e não têm valor de ordem.
14. **Browser** (categórico, códigos 1 a 13): navegador usado na sessão, também codificado.
15. **Region** (categórico, códigos 1 a 9): região geográfica de onde veio o acesso.

16. **TrafficType** (categórico, códigos 1 a 20): origem do tráfego que levou o visitante ao site, como acesso direto, mecanismo de busca, anúncio ou link de outro site.

17. **VisitorType** (texto): tipo de visitante. Pode ser *Returning_Visitor* (já visitou o site antes), *New_Visitor* (primeira visita) ou *Other* (não classificado).

Variável alvo (atributo classe)

18. **Revenue** (booleano, True/False): indica se a sessão terminou em compra ou não (2 classes existentes).

Uma observação: a documentação não diz qual sistema operacional, navegador, região ou tipo de tráfego cada código representa. Por isso, esses quatro atributos devem ser tratados como categorias, e não como números contínuos.

Principais observações da análise exploratória

- **Desbalanceamento:** 84,5% (10.422) das sessões não terminam em compra (classe negativa) e 15,5% (1.908) terminam (classe positiva), como mostra a Figura 1. Um classificador trivial que sempre responde "não comprou" já acerta 84,5%, então a acurácia sozinha não serve como medida de avaliação.
- **Valores ausentes:** nenhum.
- **Duplicatas:** 125 linhas (1,0%), todas da classe "não comprou" e todas com o mesmo padrão de sessão de saída imediata (`BounceRates` = `ExitRates` = 0,2, sem tempo em páginas de produto). Não há conflito de rótulo.
- **Inconsistências:** categoria *Other* em `VisitorType` (85 sessões), não citada na documentação; meses Jan e Apr ausentes e June fora do padrão de três letras; `SpecialDay` assume apenas 6 valores discretos (0 a 1, de 0,2 em 0,2); várias categorias raras ($< 1\%$ das sessões) em `Browser`, `OperatingSystems` e `TrafficType`.
- **Outliers:** pelo critério de Tukey, vão de 3,3% (`Administrative`) a 22,1% (`PageValues`) das sessões (Figura 3). Em `Informational`, `Informational_Duration`, `PageValues` e `SpecialDay` o IQR é 0, então todo valor diferente de zero é contado como outlier. As sessões marcadas como outlier nos atributos de navegação compram mais que a média (23% a 30%, contra 15,5%), ou seja, carregam informação e não parecem erros.
- **Atributos mais associados à classe:** `PageValues` ($\rho = 0{,}63$): quando é maior que zero, 56,3% das sessões compram; quando é zero, só 3,9%. Em seguida vêm `ExitRates` ($\rho = -0{,}25$) e os atributos de produto. Contagens e durações de um mesmo tipo de página são fortemente correlacionadas (ρ de 0,88 a 0,95), como mostra a Figura 2.
- **Categóricos:** a taxa de compra varia bastante com o mês (1,6% em fevereiro, 25,4% em novembro) e com o tipo de visitante (novos 24,9%, recorrentes 13,9%).

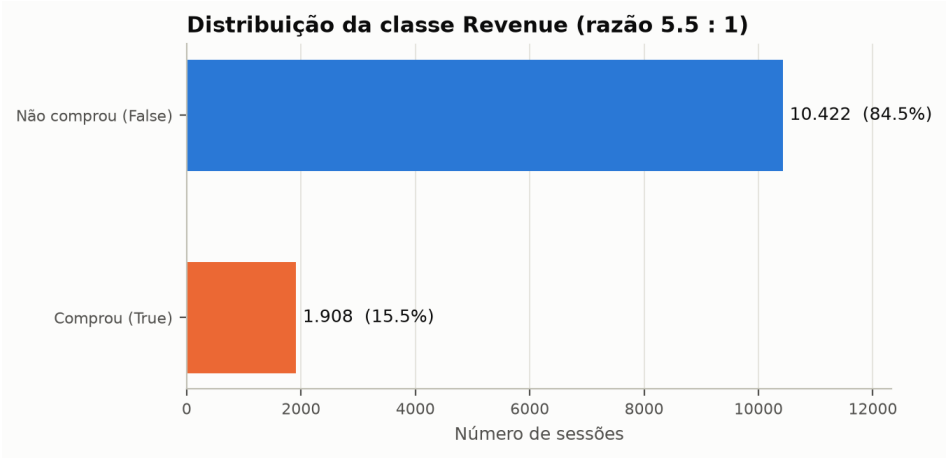


Figura 1 – Distribuição da classe `Revenue`.

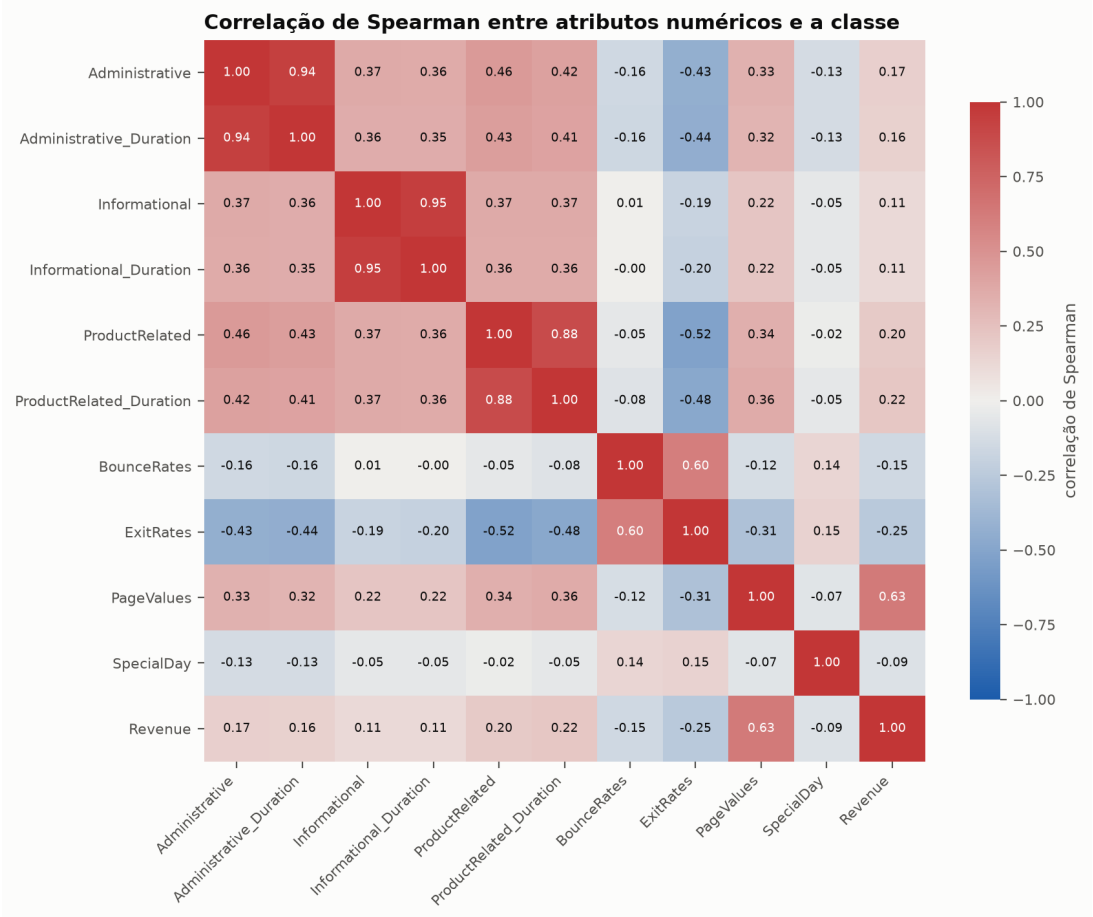


Figura 2 – Correlação de Spearman entre os atributos numéricos e a classe.

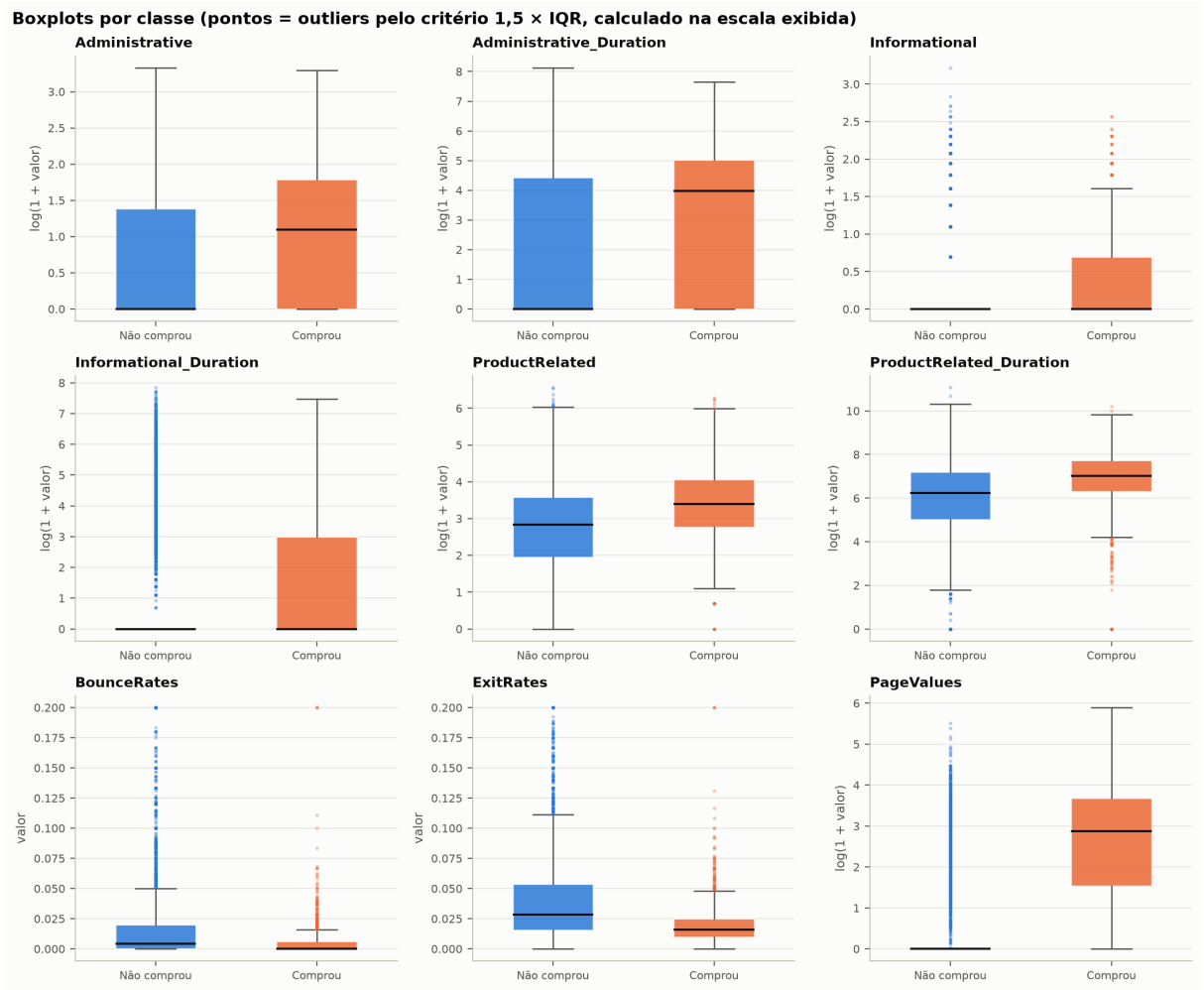


Figura 3 – Boxplots dos atributos numéricos por classe, em escala $\log(1+x)$ para os atributos assimétricos. Os pontos são outliers pelo critério de 1,5 × IQR.

III Metodologia

III.1 Pré-processamento e justificativas

- **Remover duplicatas para todos os algoritmos.** Com base na análise exploratória, verificamos que alguns registros estão duplicados. Devemos fazer sua remoção principalmente porque vamos usar o k-fold cross validation: uma cópia poderia cair no treino e outra no teste.
- **Padronizar e converter tipos, em todos os algoritmos.** O mês `June` foi padronizado para `Jun`, e `Weekend` e `Revenue` foram convertidos de True/False para 1/0. Junto com a remoção de duplicatas, essas etapas não aprendem nada com os dados e por isso foram aplicadas uma única vez, antes da divisão em treino e teste. A base resultante tem 12.205 sessões, das quais 15,63% terminam em compra.
- **Manter os outliers em todos os algoritmos.** As sessões com valores extremos compram mais que a média (23% a 30%, contra 15,5%), ou seja, os valores extremos são informativos e não erros.
- **Remover dimensões para os algoritmos KNN e MLP.** Com base no gráfico de correlações, verificamos uma correlação clara (ρ de 0,88 a 0,95) entre os pares:

  - `Administrative` e `Administrative_Duration`;
  - `Informational` e `Informational_Duration`;
  - `ProductRelated` e `ProductRelated_Duration`.

  Em cada par foi removida a contagem e mantida a duração, que carrega a mesma informação com mais detalhe. No Random Forest e no Gradient Boosting todos os atributos foram mantidos, porque a correlação entre atributos não prejudica a predição das árvores.

- **Usar $\log(1+x)$ e min-max no KNN**, para os atributos ficarem na mesma escala. O $\log(1+x)$ é aplicado antes do min-max nas durações e em `PageValues`, que são muito assimétricos: sem o log, os poucos valores extremos definiriam o máximo do min-max e a maioria das sessões ficaria comprimida perto de 0, sem diferença entre elas no cálculo da distância. As taxas (`BounceRates`, `ExitRates` e `SpecialDay`) já estão entre 0 e 1 e recebem só o min-max.

- **Usar padronização (z-score) no MLP**, para os atributos numéricos ficarem centrados (média) em 0 e com desvio padrão 1. A rede aprende por gradiente, e entradas em escalas muito diferentes deixam o treino instável e lento: por exemplo, `ProductRelated_Duration` chega a 63.973, enquanto `ExitRates` não passa de 0,2.

- **Usar one-hot encoding no KNN e no MLP** para converter os atributos categóricos em numéricos, já que esses algoritmos fazem cálculos (distâncias e somas ponderadas) que só lidam com números.

- **Agrupar categorias raras no KNN e no MLP**, com o objetivo de reduzir o ruído. Foram consideradas raras as categorias presentes em menos de 1% das sessões de treino: os sistemas operacionais 5 a 8; os navegadores 3, 7, 9, 11, 12 e 13; os tipos de tráfego 7, 9, 12, 14, 15, 16, 18 e 19; e o tipo de visitante *Other*. Cada atributo ganha uma única coluna para as suas categorias raras. Após o one-hot, o KNN e o MLP recebem 55 colunas: 47 vindas dos atributos categóricos, 7 numéricas e `Weekend`.

- **Manter os atributos categóricos como códigos inteiros no Random Forest.** `OperatingSystems`, `Browser`, `Region` e `TrafficType` já são inteiros; `Month` e `VisitorType` precisam ser convertidos em códigos. As árvores dividem os dados por limiares e não calculam distâncias, então os códigos funcionam e ainda permitem separar grupos de categorias. Com one-hot, o `TrafficType` viraria 20 colunas binárias, cada uma raramente escolhida para divisão.

- **Usar os atributos categóricos de forma nativa no Gradient Boosting.** O `HistGradientBoostingClassifier` do scikit-learn aceita atributos categóricos diretamente: em cada divisão, ele separa as categorias em dois grupos, sem precisar de one-hot e sem impor uma ordem artificial aos códigos.

- **Tratar o desbalanceamento em todos os algoritmos.** Com base na análise exploratória, 84,5% das sessões são "não comprou" e 15,5% são "comprou". Sem tratamento, os algoritmos tendem a favorecer a classe majoritária.

  - **KNN:** sobreamostragem da classe minoritária, aplicada só nos folds de treino do k-fold. O KNN não aceita pesos por classe, e com 84,5% de "não comprou" os vizinhos mais próximos tendem a ser dessa classe. Foi usada a sobreamostragem aleatória (`RandomOverSampler`), que repete sessões da classe minoritária até as duas classes ficarem com o mesmo tamanho. O SMOTE comum foi descartado porque interpolaria as colunas one-hot e criaria valores sem sentido, como 0,4; o SMOTENC, que trata atributos categóricos, fica como possível melhoria.

  - **Random Forest:** `class_weight="balanced"`, que dá a cada classe um peso inversamente proporcional à sua frequência. Assim, errar uma sessão "comprou" pesa cerca de 5,4 vezes mais, sem alterar os dados.
  - **Gradient Boosting:** `class_weight="balanced"`, pelo mesmo motivo do Random Forest.
  - **MLP:** sobreamostragem da classe minoritária, aplicada só nos folds de treino, já que o `MLPClassifier` do scikit-learn não aceita pesos por classe.

- **Evitar vazamento de informação.** Todas as etapas que aprendem algo com os dados (min-max, z-score, agrupamento de categorias raras e sobreamostragem) foram montadas em um `Pipeline` da biblioteca imbalanced-learn e são ajustadas somente na parte de treino de cada fold da validação cruzada. A parte de avaliação nunca é sobreamostrada e mantém a proporção real das classes.

A Tabela 1 resume o pré-processamento aplicado em cada algoritmo.

Tabela 1 – Pré-processamento por algoritmo.

Etapa	KNN	Random Forest	Gradient Boosting	MLP
Remover duplicatas, June → Jun, booleanos → 0/1	sim	sim	sim	sim
Remover contagens redundantes	sim	não	não	sim
$\log(1+x)$	sim	não	não	não
Escala	min-max	não	não	z-score
Atributos categóricos	one-hot	códigos inteiros	categóricos nativos	one-hot
Agrupar categorias raras	sim	não	não	sim
Desbalanceamento	sobreamostragem	`class_weight`	`class_weight`	sobreamostragem

III.2 Ferramentas

Todo o trabalho foi feito em Python 3.13, em notebooks Jupyter, dentro de um ambiente virtual gerenciado pelo uv, que fixa as versões das bibliotecas e permite reproduzir os resultados. O código foi organizado em três notebooks, um por fase: análise exploratória, pré-processamento e classificação. As bibliotecas usadas foram:

- **pandas** 3.1.1 e **NumPy** 2.5.4: leitura e manipulação dos dados;
- **Matplotlib** 3.11.2: gráficos;
- **scikit-learn** 1.9.1: algoritmos de classificação, transformações (min-max, z-score, one-hot, códigos inteiros), pipelines, validação cruzada, busca em grade e medidas de avaliação;
- **imbalanced-learn** 0.14.2: sobreamostragem aleatória (`RandomOverSampler`) e o `Pipeline` que aplica a sobreamostragem apenas nos dados de treino.

Em todos os passos aleatórios (divisão dos dados, folds, sobreamostragem e inicialização dos modelos) foi usada a semente 42.

III.3 Algoritmos de classificação

Foram usados quatro algoritmos. Os parâmetros testados para cada algoritmo estão na Tabela 2.

1) KNN (k vizinhos mais próximos)

Funcionamento. O KNN não constrói um modelo durante o treino: ele apenas guarda as sessões de treino. Para classificar uma sessão nova, calcula a distância dela até todas as sessões de treino, seleciona as k mais próximas e atribui a classe mais votada entre elas. O voto pode ser uniforme ou ponderado pelo inverso da distância, e a probabilidade de compra é a fração (ponderada) de vizinhos compradores. A distância usada é a de Minkowski, que com $p = 1$ é a distância de Manhattan e com $p = 2$ é a euclidiana. Os dois valores de p foram testados na busca em grade (Tabela 2), e o modelo final usa a distância de Manhattan ($p = 1$), que obteve F1 maior que a euclidiana em todos os valores de k testados (Seção IV.1).

Justificativa. É um classificador simples, não paramétrico e sem suposições sobre a forma da fronteira de decisão, o que o torna uma boa referência. Como depende de distâncias, também mostra o efeito do pré-processamento (escala, assimetria e codificação dos atributos categóricos) sobre o resultado.

Parâmetros. O número de vizinhos k controla o equilíbrio entre um modelo que segue o ruído (k pequeno) e um modelo que suaviza demais a fronteira (k grande). O tipo de peso e a distância mudam quanto cada vizinho e cada atributo influenciam a decisão.

Pré-processamento necessário. Remoção das contagens redundantes, $\log(1 + x)$ e min-max nos atributos numéricos, one-hot com agrupamento de categorias raras e sobreamostragem. O KNN exige atributos numéricos e na mesma escala, pois um atributo com valores grandes dominaria a distância.

2) Random Forest

Funcionamento. O Random Forest (Breiman, 2001) é um conjunto (*ensemble*) de árvores de decisão cujas previsões são combinadas por votação. Uma árvore de decisão divide as sessões de forma recursiva: em cada nó, escolhe um atributo e um limiar (por exemplo, `PageValues` $\leq 0{,}5$) que separam as sessões em dois grupos mais homogêneos quanto à classe, e repete o processo em cada grupo até atingir um critério de parada. A homogeneidade é medida pela impureza de Gini,

$$\text{Gini}(D) = 1 - p_1^2 - p_0^2,$$

em que p_1 e p_0 são as proporções de sessões com e sem compra no grupo D. A melhor divisão é a que mais reduz a impureza média dos dois grupos resultantes, e cada folha guarda a proporção de compradores das sessões que caíram nela.

Uma única árvore profunda decora o treino e varia muito com pequenas mudanças nos dados. O Random Forest reduz essa variância com duas fontes de aleatoriedade: (1) cada árvore é treinada com uma amostra *bootstrap*, isto é, n sessões sorteadas com reposição do treino; e (2) em cada nó, a melhor divisão é procurada apenas entre m atributos sorteados (`max_features`), e não entre todos. Assim, as árvores ficam diferentes entre si e seus erros se compensam. A probabilidade de compra prevista é a média das probabilidades dadas por todas as árvores, e a sessão é classificada como compra se essa média for pelo menos 0,5.

Com `class_weight="balanced"`, cada sessão recebe um peso inversamente proporcional à frequência da sua classe (cerca de 3,2 para as sessões com compra e 0,59 para as demais). Os pesos entram no cálculo da impureza de Gini e das proporções nas folhas, de modo que errar uma sessão com compra pesa cerca de 5,4 vezes mais. O pseudocódigo está no Algoritmo 1.

Algoritmo 1 Random Forest para classificação binária

Entrada: treino $D = \{(x_i, y_i)\}_{i=1}^{n}$ com pesos por classe w_i; número de árvores B (`n_estimators`); atributos por divisão m (`max_features`); `max_depth`; `min_samples_leaf`

para $b = 1$ **até** B **faça**
  $D_b \leftarrow n$ sessões sorteadas com reposição de D ▷ *amostra bootstrap*
  $T_b \leftarrow$ ConstruirÁrvore$(D_b, 0)$
fim para

Saída: $P(\text{comprou} \mid x) = \frac{1}{B}\sum_{b=1}^{B} P_b(\text{comprou} \mid x)$; classe = comprou se $P \geq 0{,}5$

função ConstruirÁrvore(D, *prof*)
  se D tem uma só classe **ou** *prof* = `max_depth` **ou** $|D| < 2 \cdot$ `min_samples_leaf` **então**
    retornar folha com $P(\text{comprou})$ = peso dos compradores em D / peso total de D
  fim se
  $A \leftarrow m$ atributos sorteados sem reposição
  para cada atributo $a \in A$ e limiar t entre valores consecutivos de a em D **faça**
    $D_E \leftarrow \{x \in D : x_a \leq t\}$; $D_D \leftarrow D \setminus D_E$
    $\Delta(a,t) \leftarrow \text{Gini}(D) - \frac{|D_E|}{|D|}\text{Gini}(D_E) - \frac{|D_D|}{|D|}\text{Gini}(D_D)$ ▷ *tamanhos ponderados por* w_i
  fim para
  $(a^*, t^*) \leftarrow \arg\max \Delta(a,t)$
  $T_E \leftarrow$ ConstruirÁrvore$(D_E, prof+1)$; $T_D \leftarrow$ ConstruirÁrvore$(D_D, prof+1)$
  retornar nó com o teste $x_{a^*} \leq t^*$, filho esquerdo T_E e filho direito T_D
fim função

Justificativa. Lida bem com atributos de tipos diferentes, com relações não lineares e com interações entre atributos, não exige normalização e não é afetado pela assimetria nem pelos outliers, pois as árvores dividem os dados por limiares. Também aceita pesos por classe para tratar o desbalanceamento e é pouco propenso a sobreajuste, por fazer a média de muitas árvores.

Parâmetros. O número de árvores, a quantidade de atributos sorteados em cada divisão, a profundidade máxima e o mínimo de instâncias por folha. Os dois últimos limitam o quanto cada árvore pode decorar o treino.

Pré-processamento necessário. Apenas a conversão de `Month` e `VisitorType` em códigos inteiros. Os demais atributos são usados como estão, e o desbalanceamento é tratado com `class_weight="balanced"`.

3) Gradient Boosting

Funcionamento. O Gradient Boosting (Friedman, 2001) também combina árvores de decisão, mas de forma sequencial: cada nova árvore é treinada para corrigir os erros do conjunto formado pelas árvores anteriores. O modelo é uma soma

$$F(x) = F_0 + \eta\, T_1(x) + \eta\, T_2(x) + \cdots + \eta\, T_M(x),$$

em que cada T_m é uma árvore de regressão pequena e η é a taxa de aprendizado. Para classificação binária, $F(x)$ é o logaritmo da razão de chances de compra, e a probabilidade é obtida pela função sigmoide, $P(\text{comprou} \mid x) = 1/(1 + e^{-F(x)})$.

Em cada iteração, o algoritmo calcula, para cada sessão de treino, o gradiente da perda logística em relação à previsão atual ($g = p - y$, a diferença entre a probabilidade prevista e a classe real) e a curvatura dessa perda ($h = p(1-p)$). A nova árvore é construída para seguir a direção oposta ao gradiente, ou seja, para reduzir a perda onde o modelo mais erra. A taxa de aprendizado η reduz o tamanho de cada passo, o que torna o modelo mais estável e menos propenso a sobreajuste, mas exige mais árvores.

A implementação usada foi o `HistGradientBoostingClassifier` do scikit-learn, inspirado no LightGBM (Ke et al., 2017). Ele discretiza cada atributo numérico em no máximo 255

faixas (histogramas), o que torna a busca da melhor divisão muito mais rápida, e trata atributos categóricos de forma nativa, separando as categorias em dois grupos a cada divisão. Os pesos por classe (`class_weight`) multiplicam o gradiente e a curvatura de cada sessão, de modo que os erros nas sessões com compra pesam mais. O pseudocódigo está no Algoritmo 2.

Algoritmo 2 Gradient Boosting com histogramas para classificação binária

Entrada: treino $\{(x_i, y_i)\}_{i=1}^{n}$, $y_i \in \{0,1\}$, com pesos por classe w_i; número de árvores M (`max_iter`); taxa de aprendizado η; limites da árvore (`max_depth`, máximo de 31 folhas); regularização λ

discretizar cada atributo numérico em até 255 faixas (histogramas)
$p \leftarrow$ proporção ponderada de compradores no treino
$F_0(x) \leftarrow \log\big(p/(1-p)\big)$ ▷ *previsão inicial igual para todos*
para $m = 1$ **até** M **faça**
  para cada sessão i **faça**
    $p_i \leftarrow 1/\big(1 + e^{-F_{m-1}(x_i)}\big)$ ▷ *probabilidade atual*
    $g_i \leftarrow w_i\,(p_i - y_i)$ ▷ *gradiente da perda logística*
    $h_i \leftarrow w_i\,p_i\,(1 - p_i)$ ▷ *curvatura (2ª derivada)*
  fim para
  $T_m \leftarrow$ nova árvore com todas as sessões na raiz
  enquanto houver folha que possa ser dividida respeitando os limites **faça**
    para cada atributo e fronteira entre faixas (ou grupo de categorias) **faça**
      $G_E, H_E \leftarrow$ somas de g e h à esquerda; $G_D, H_D \leftarrow$ somas à direita
      ganho $\leftarrow \frac{G_E^2}{H_E+\lambda} + \frac{G_D^2}{H_D+\lambda} - \frac{(G_E+G_D)^2}{H_E+H_D+\lambda}$
    fim para
    dividir a folha pela fronteira de maior ganho
  fim enquanto
  para cada folha j de T_m **faça**
    $v_j \leftarrow -\sum_{i\in j} g_i \,/\, \big(\sum_{i\in j} h_i + \lambda\big)$
  fim para
  $F_m(x) \leftarrow F_{m-1}(x) + \eta\, T_m(x)$ ▷ $T_m(x)$ *= valor da folha em que x cai*
fim para

Saída: $P(\text{comprou} \mid x) = 1/\big(1 + e^{-F_M(x)}\big)$; classe = comprou se $P \geq 0{,}5$

Justificativa. O Gradient Boosting está entre os algoritmos com melhor desempenho em dados tabulares. Enquanto o Random Forest reduz a variância fazendo a média de árvores independentes, o boosting reduz o viés corrigindo erros de forma sequencial, o que permite comparar as duas estratégias de combinação de árvores. Assim como o Random Forest, não exige normalização e não é afetado pela assimetria dos atributos.

Parâmetros. A taxa de aprendizado η, o número de árvores M e a profundidade máxima de cada árvore. Taxas maiores com muitas árvores tendem a sobreajustar; taxas menores precisam de mais árvores.

Pré-processamento necessário. Apenas a conversão dos atributos categóricos em códigos para o tratamento nativo. O desbalanceamento é tratado com `class_weight="balanced"`.

4) Rede neural MLP – Multilayer Perceptron

Funcionamento. O MLP é uma rede neural formada por camadas de neurônios totalmente conectadas. Neste trabalho a rede tem três camadas: a de entrada, com uma posição para cada uma das 55 colunas produzidas pelo pré-processamento; uma camada oculta com H neurônios; e a de saída, com um único neurônio. Cada neurônio oculto calcula uma soma ponderada das entradas, soma um viés e aplica uma função de ativação não linear φ (ReLU, $\varphi(z) = \max(0, z)$, ou tangente hiperbólica). O neurônio de saída faz o mesmo com as saídas da camada oculta e aplica a sigmoide, produzindo a probabilidade de compra. É a camada oculta não linear que permite à rede aprender fronteiras de decisão complexas.

O treino ajusta os pesos para minimizar a perda logística (entropia cruzada) somada a uma penalidade L2, $\alpha\|W\|^2$, que desestimula pesos grandes e reduz o sobreajuste. O gradiente da perda em relação a cada peso é calculado pela retropropagação (Rumelhart, Hinton e Williams, 1986), que aplica a regra da cadeia da saída para a entrada. Os pesos são atualizados pelo otimizador Adam (Kingma e Ba, 2015) em lotes de 200 sessões, e o treino para quando a perda deixa de melhorar por 10 épocas seguidas ou ao atingir o limite de épocas. O pseudocódigo está no Algoritmo 3.

Algoritmo 3 Treino do MLP com uma camada oculta

Entrada: treino padronizado e balanceado $\{(x_i, y_i)\}_{i=1}^{n}$, $x_i \in \mathbb{R}^{55}$; H neurônios ocultos; ativação φ (ReLU ou tanh); regularização α; taxa $\eta = 0{,}001$; lote $B = 200$; limite de 1000 épocas

inicializar $W_1 \in \mathbb{R}^{55\times H}$, b_1, $W_2 \in \mathbb{R}^{H\times 1}$, b_2 com valores aleatórios pequenos
para época $= 1$ **até** 1000 **faça**
  embaralhar o treino e dividi-lo em lotes de B sessões
  para cada lote (X, y) **faça**
    $A \leftarrow \varphi(XW_1 + b_1)$ ▷ *camada oculta*
    $\hat{y} \leftarrow 1/\big(1 + e^{-(AW_2+b_2)}\big)$ ▷ *probabilidade de compra*
    $L \leftarrow -\,\text{média}\,\big[y \log \hat{y} + (1-y)\log(1-\hat{y})\big] + \frac{\alpha}{2n}\big(\|W_1\|^2 + \|W_2\|^2\big)$
    $\delta_2 \leftarrow \hat{y} - y$ ▷ *erro na saída*
    $\nabla W_2 \leftarrow A^\top \delta_2 / B + \frac{\alpha}{n} W_2; \quad \nabla b_2 \leftarrow \text{média}(\delta_2)$
    $\delta_1 \leftarrow (\delta_2 W_2^\top) \odot \varphi'(XW_1 + b_1)$ ▷ *erro retropropagado*
    $\nabla W_1 \leftarrow X^\top \delta_1 / B + \frac{\alpha}{n} W_1; \quad \nabla b_1 \leftarrow \text{média}(\delta_1)$
    atualizar W_1, b_1, W_2, b_2 com o Adam usando os gradientes
  fim para
  se a perda não melhorar mais que 10^{-4} por 10 épocas seguidas **então**
    parar
  fim se
fim para

Saída: $P(\text{comprou} \mid x) = \hat{y}(x)$; classe = comprou se $P \geq 0{,}5$

Justificativa. As redes neurais conseguem aproximar relações não lineares complexas e são uma abordagem diferente das árvores, o que enriquece a comparação. Além disso, o artigo que publicou esta base (Sakar et al., 2019) usou justamente um MLP para prever a intenção de compra.

Parâmetros. O número de neurônios da camada oculta (capacidade da rede), a força da regularização α e a função de ativação. Com poucos dados e muitos parâmetros, a regularização é o que impede a rede de decorar o treino.

Pré-processamento necessário. Remoção das contagens redundantes, z-score nos atributos numéricos, one-hot com agrupamento de categorias raras e sobreamostragem. A rede aprende por gradiente, e atributos em escalas muito diferentes deixariam o treino instável e lento.

III.4 Parâmetros testados

Os parâmetros não foram fixados previamente. Para cada algoritmo, os principais parâmetros receberam uma lista de valores, e todas as combinações foram avaliadas por busca em grade (`GridSearchCV`), totalizando 72 combinações (Tabela 2). Os valores cobrem desde modelos mais simples até modelos mais complexos, para mostrar como cada parâmetro afeta o resultado.

Tabela 2 – Grade de parâmetros testada em cada algoritmo.

Algoritmo	Parâmetro	Valores testados	O que controla
KNN	`n_neighbors`	5, 15, 31, 51, 101	Número de vizinhos que votam
(20 combinações)	`weights`	uniform, distance	Voto igual ou ponderado pela distância
	`p`	1 (Manhattan), 2 (euclidiana)	Distância usada
Random Forest	`n_estimators`	100, 300	Número de árvores
(16 combinações)	`max_features`	sqrt, 0,5	Atributos sorteados em cada divisão
	`max_depth`	sem limite, 10	Profundidade máxima das árvores
	`min_samples_leaf`	1, 5	Mínimo de instâncias por folha
Gradient Boosting	`learning_rate`	0,05; 0,1; 0,2	Tamanho do passo de cada árvore
(18 combinações)	`max_iter`	100, 300	Número de árvores
	`max_depth`	sem limite, 3, 6	Profundidade de cada árvore
MLP	`hidden_layer_sizes`	10, 50, 100 neurônios	Tamanho da camada oculta
(18 combinações)	`alpha`	0,0001; 0,01; 1	Força da regularização L2
	`activation`	relu, tanh	Função de ativação

Alguns valores foram mantidos fixos: a semente aleatória (42), para tornar os resultados reprodutíveis; o tratamento do desbalanceamento (`class_weight` ou sobreamostragem), que faz parte do pré-processamento; e o limite de 1000 épocas do MLP, que é um critério de parada e não um parâmetro do modelo, já que o treino para antes quando a perda deixa de melhorar. Os demais parâmetros usam os valores padrão do scikit-learn.

III.5 Estratégia de avaliação e preparação da base

A avaliação foi planejada para que os quatro algoritmos fossem comparados nas mesmas condições e para que a escolha dos parâmetros não usasse os dados em que o resultado final é medido. A base limpa (12.205 sessões) foi preparada em três passos:

1. **Divisão em treino e teste.** A base foi dividida uma única vez em 80% de treino e 20% de teste, de forma estratificada, mantendo a mesma proporção de compradores nas duas partes (Tabela 3). O teste foi guardado e só foi usado no final, uma única vez.

2. **Validação cruzada no treino para testar as parametrizações.** Cada combinação de parâmetros foi avaliada com validação cruzada estratificada de 10 folds (`StratifiedKFold`, com embaralhamento): o treino é dividido em 10 partes com a mesma proporção de compradores, e em cada rodada 9 partes treinam o modelo e 1 parte o avalia. Os **mesmos 10 folds** foram usados para todos os algoritmos e combinações, então as diferenças observadas vêm dos modelos e não de divisões diferentes dos dados. O pipeline completo (pré-processamento, sobreamostragem e modelo) é reajustado do zero em cada fold. O resultado é a média dos 10 folds, com o desvio padrão indicando o quanto ele varia conforme os dados. No total, foram 72 combinações $\times$ 10 folds = 720 treinos, além de duas linhas de base que não aprendem nada (uma que sempre responde "não comprou" e outra que sorteia a resposta), usadas como referência.

3. **Escolha e avaliação final.** Para cada algoritmo, a melhor parametrização é a de maior F1 médio na validação cruzada. Ela é treinada com todo o treino e avaliada no teste, que fornece as medidas finais, as matrizes de confusão e a tabela comparativa.

A validação cruzada foi usada além da divisão em treino e teste porque um único teste, com 382 compradores, tem variação considerável: o desvio padrão do F1 entre os folds ficou

em torno de 0,03, maior que a diferença entre várias parametrizações. A média de 10 folds dá uma estimativa mais estável para comparar parametrizações, e o teste separado confirma se essa estimativa se mantém em dados nunca usados nas escolhas.

Tabela 3 – Divisão estratificada da base limpa em treino e teste.

Conjunto	Sessões	Não comprou	Comprou	% comprou
Base limpa	12.205	10.297	1.908	15,63%
Treino (80%)	9.764	8.238	1.526	15,63%
Teste (20%)	2.441	2.059	382	15,65%

III.6 Medidas de avaliação

A classe positiva é "comprou". Comparando a classe prevista com a real, cada sessão do teste cai em uma das quatro células da matriz de confusão: **VP** (verdadeiro positivo, comprador identificado), **FN** (falso negativo, comprador perdido), **FP** (falso positivo, alarme falso) e **VN** (verdadeiro negativo, não-comprador identificado). As medidas usadas são:

Precisão: das sessões previstas como compra, a fração que de fato comprou.

$$\text{Precisão} = \frac{VP}{VP + FP}$$

Revocação (sensibilidade): dos compradores reais, a fração que o modelo encontrou.

$$\text{Revocação} = \frac{VP}{VP + FN}$$

F1: média harmônica entre precisão e revocação da classe "comprou".

$$F_1 = \frac{2 \cdot \text{Precisão} \cdot \text{Revocação}}{\text{Precisão} + \text{Revocação}} = \frac{2\,VP}{2\,VP + FP + FN}$$

PR-AUC (precisão média): área sob a curva de precisão × revocação, obtida variando o limiar de decisão. Ordenando as sessões pela probabilidade de compra, P_n e R_n são a precisão e a revocação no n-ésimo limiar. Um modelo sem conhecimento atinge a proporção de compradores ($\approx 0{,}156$).

$$\text{PR-AUC} = \sum_n (R_n - R_{n-1})\, P_n$$

Escolha das medidas. As quatro medidas acima se concentram na classe "comprou", que é a de interesse do problema. Duas medidas muito usadas foram descartadas, e os resultados da validação cruzada mostram por quê:

- **Acurácia** (fração de todas as sessões classificadas corretamente). Com 84,4% das sessões sem compra, ela é enganosa: a linha de base que sempre responde "não comprou" atingiu 84,4% de acurácia sem encontrar nenhum comprador (F1 = 0). Além disso, na busca em grade a acurácia escolheria, em três dos quatro algoritmos, a parametrização que encontra menos compradores. No MLP, por exemplo, ela escolheria uma rede sem regularização que encontra 57% dos compradores, contra 80% da escolhida pelo F1.
- **ROC-AUC** (área sob a curva da revocação contra a taxa de falsos positivos, TFP = $FP/(FP + VN)$, variando o limiar). Assim como a PR-AUC, mede a qualidade da ordenação das sessões, mas é otimista com classes desbalanceadas: o denominador da TFP é dominado pelos milhares de não-compradores, então muitos alarmes falsos ainda resultam

em uma taxa baixa. Na validação cruzada, pela ROC-AUC o KNN (0,850) percorreu 70% do caminho entre o acaso (0,5) e o modelo perfeito, contra 86% do Random Forest, e parece não estar tão distante dele. Pela PR-AUC, o KNN percorreu só 44% do caminho, contra 70% do Random Forest. A PR-AUC usa a precisão, que cai assim que os alarmes falsos se acumulam, e por isso mostra a diferença real entre os modelos.

Assim, as medidas ficaram com os papéis da Tabela 4.

Tabela 4 – Papel de cada medida na avaliação.

Medida	Papel	Justificativa
F1 (classe "comprou")	Principal: escolhe a parametrização e compara os algoritmos	Só é alta se o modelo encontra os compradores e acerta quando prevê uma compra. Mede a decisão real do modelo (limiar 0,5), a mesma da matriz de confusão.
PR-AUC	Segunda medida	Avalia a ordenação das sessões em todos os limiares, focada na classe "comprou", e não fica inflada pelo desbalanceamento.
Precisão e revocação	Complementares	Mostram se o modelo erra mais por alarmes falsos ou por compradores perdidos.
Matriz de confusão	Complementar	Mostra os quatro tipos de acerto e erro em números absolutos.
Acurácia	Não usada	Enganosa com classes desbalanceadas: premia o modelo que nunca prevê uma compra.
ROC-AUC	Não usada	Otimista com classes desbalanceadas; a PR-AUC mede a mesma ordenação sem esse problema.

IV Resultados e discussão

IV.1 Resultados da busca em grade (validação cruzada no treino)

As linhas de base definem o ponto de partida: a que sempre responde "não comprou" tem F1 = 0 e 84,4% de acurácia, e a que sorteia a resposta tem F1 = 0,154 e PR-AUC = 0,156. Todos os algoritmos ficaram muito acima dessas referências. A Figura 4 mostra o efeito de cada parâmetro no F1 médio, e a Tabela 5 mostra quanto o F1 variou entre a melhor e a pior combinação de cada algoritmo.

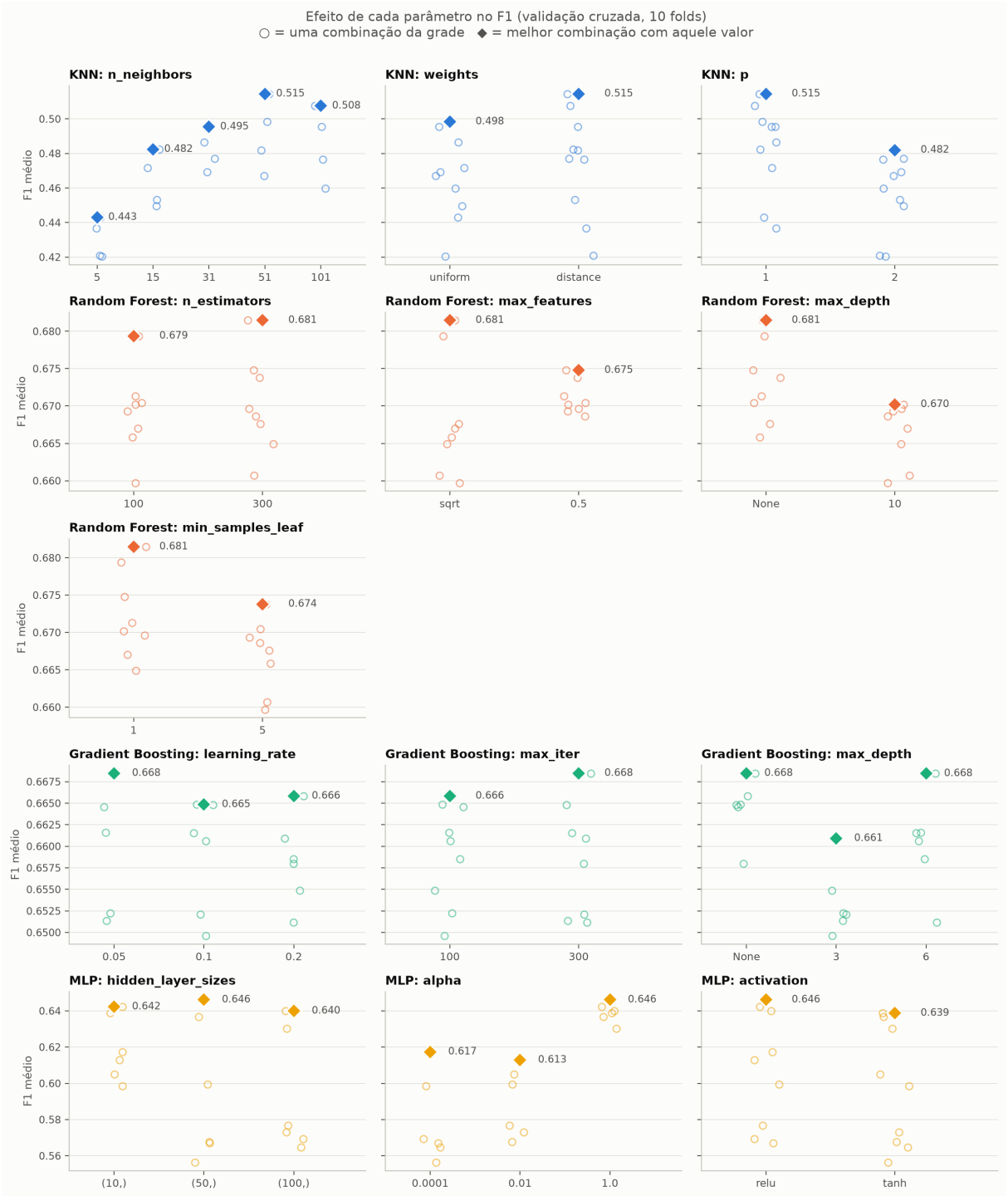


Figura 4 – Efeito de cada parâmetro no F1 médio da validação cruzada (10 folds). Cada círculo é uma combinação da grade, e o losango é a melhor combinação com aquele valor do parâmetro.

Tabela 5 – Sensibilidade de cada algoritmo aos parâmetros (F1 médio na validação cruzada).

Algoritmo	Combinações	Pior F1	Melhor F1	Amplitude	Desvio padrão entre folds
KNN	20	0,420	0,515	0,094	0,021
Random Forest	16	0,660	0,681	0,022	0,030
Gradient Boosting	18	0,650	0,668	0,019	0,032
MLP	18	0,556	0,646	0,090	0,032

No KNN e no MLP, a amplitude é cerca de três vezes o desvio padrão entre os folds: os parâmetros mudam o resultado de fato. No MLP, a regularização foi decisiva: todas as combinações com $\alpha = 1$ ficaram entre 0,630 e 0,646 de F1, enquanto as redes de 50 e 100 neurônios com α pequeno caíram para 0,556 a 0,599, porque decoram o treino. Com regularização forte, o tamanho da camada e a função de ativação pesam pouco. No KNN, o F1 melhora com mais vizinhos até $k = 51$ (de 0,443 com $k = 5$ para 0,515), a distância de Manhattan supera a euclidiana em todos os valores de k e o peso por distância ajuda um pouco. No Random Forest e no Gradient Boosting, as combinações ficam numa faixa menor que o desvio padrão entre os folds, ou seja, esses algoritmos são pouco sensíveis aos parâmetros testados e os valores padrão já funcionam bem.

A Tabela 6 mostra a melhor parametrização de cada algoritmo, escolhida pelo maior F1 médio.

Tabela 6 – Melhor parametrização de cada algoritmo na validação cruzada (média ± desvio padrão dos 10 folds).

Algoritmo	Melhor parametrização	F1	PR-AUC	Precisão	Revocação
KNN	$k = 51$, peso por distância, Manhattan ($p = 1$)	0,515 ± 0,018	0,514 ± 0,030	0,392	0,749
Random Forest	300 árvores, `max_features` = sqrt, sem limite de profundidade, folha ≥ 1	0,681 ± 0,036	0,738 ± 0,032	0,634	0,737
Gradient Boosting	`learning_rate` = 0,05, 300 árvores, sem limite de profundidade	0,668 ± 0,034	0,739 ± 0,034	0,602	0,753
MLP	50 neurônios, $\alpha = 1$, ReLU	0,646 ± 0,031	0,700 ± 0,037	0,541	0,804

IV.2 Resultados no conjunto de teste

A Tabela 7 é a tabela comparativa final, com as quatro medidas escolhidas calculadas no teste para a melhor parametrização de cada algoritmo. A coluna "F1 (val. cruzada)" repete a estimativa da validação cruzada: em todos os algoritmos, o F1 no teste ficou a no máximo 0,02 dela, o que confirma que a escolha dos parâmetros não foi sobreajustada.

Tabela 7 – Tabela comparativa no conjunto de teste (2.441 sessões, 382 com compra), ordenada pelo F1. Em negrito, o melhor valor de cada medida.

Algoritmo	F1	PR-AUC	Precisão	Revocação	F1 (val. cruzada)
Random Forest	**0,683**	**0,739**	**0,633**	0,741	0,681
Gradient Boosting	0,666	0,727	0,588	0,767	0,668
MLP	0,655	0,710	0,549	**0,814**	0,646
KNN	0,535	0,555	0,403	0,796	0,515

As matrizes de confusão (Figura 5 e Tabela 8) mostram como cada algoritmo erra.

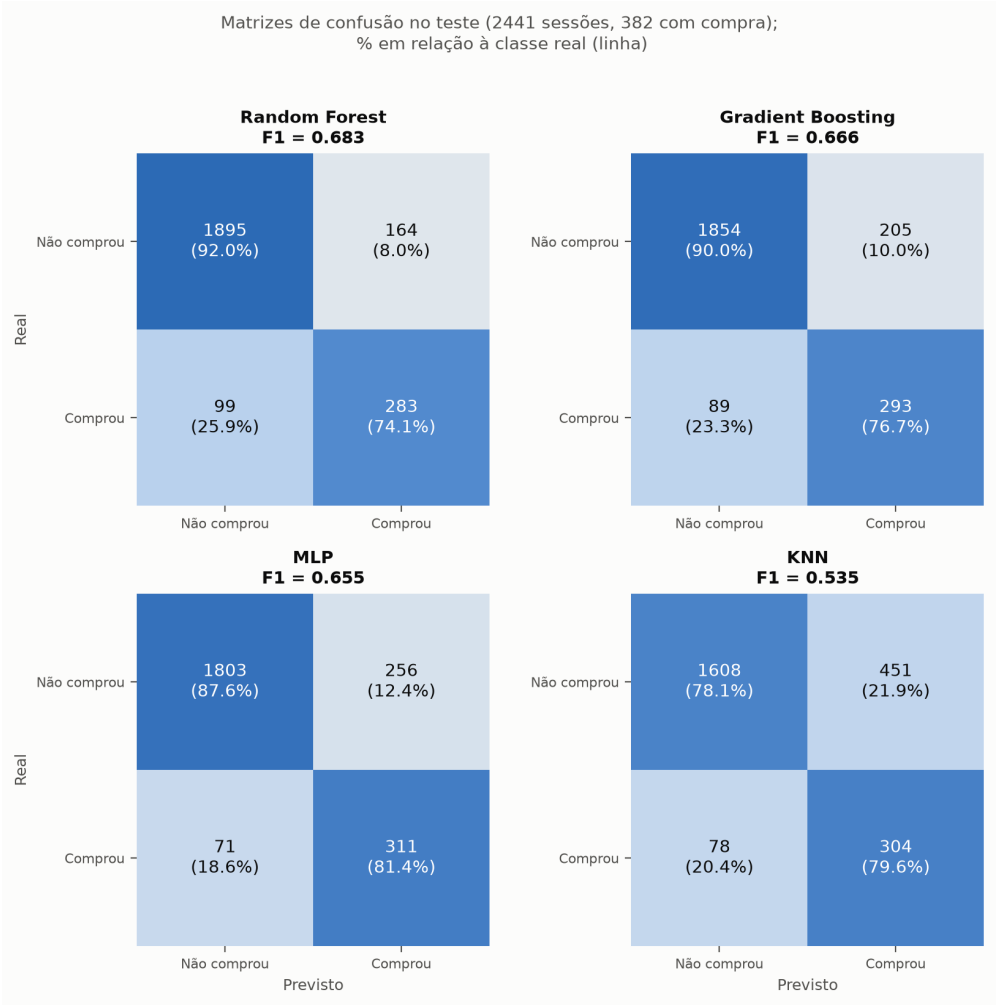


Figura 5 – Matrizes de confusão no conjunto de teste. As porcentagens são relativas à classe real (linha).

Tabela 8 – Células das matrizes de confusão no teste.

Algoritmo	VN	FP (alarme falso)	FN (comprador perdido)	VP (comprador encontrado)
Random Forest	1.895	164	99	283 (74,1%)
Gradient Boosting	1.854	205	89	293 (76,7%)
MLP	1.803	256	71	311 (81,4%)
KNN	1.608	451	78	304 (79,6%)

A Figura 6 mostra as curvas de precisão × revocação, cuja área é a PR-AUC. O ponto em cada curva é o limiar padrão de 0,5, que gera as matrizes de confusão.

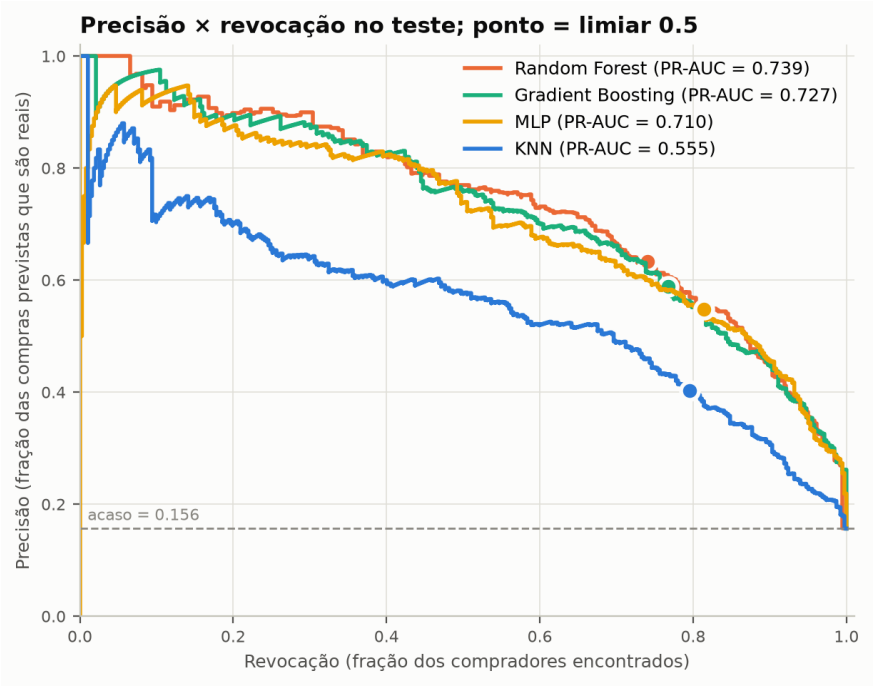


Figura 6 – Curvas de precisão × revocação no teste. O ponto marca o limiar de 0,5, e a linha tracejada é o desempenho de um classificador sem conhecimento.

IV.3 Discussão

Qual algoritmo apresentou o melhor desempenho? A resposta depende da medida? O Random Forest foi o melhor pelas duas medidas principais (F1 = 0,683 e PR-AUC = 0,739) e também pela precisão (0,633). A resposta, porém, depende da medida: pela revocação, o melhor seria o MLP (0,814), porque ele prevê mais compras e, assim, encontra mais compradores, à custa de mais alarmes falsos. É por isso que a decisão foi tomada pelo F1, que equilibra precisão e revocação. Além disso, a diferença entre Random Forest e Gradient Boosting (0,017 de F1) é menor que o desvio padrão entre os folds, então os dois estão praticamente empatados; o KNN ficou claramente atrás em todas as medidas, exceto a revocação.

A alteração dos parâmetros modificou os resultados? Sim, mas de forma muito diferente entre os algoritmos (Tabela 5 e Figura 4). O KNN e o MLP mudaram bastante (amplitude de 0,094 e 0,090 de F1), com destaque para o número de vizinhos e a distância no KNN e para a regularização no MLP. O Random Forest e o Gradient Boosting variaram menos que o desvio padrão entre os folds: são robustos à escolha dos parâmetros.

Os algoritmos apresentaram comportamentos diferentes em relação aos erros? Sim (Tabela 8). O Random Forest é o mais conservador: gera menos alarmes falsos (164), mas perde mais compradores (99). O MLP é o oposto: perde menos compradores (71), mas gera 256 alarmes falsos. O Gradient Boosting fica entre os dois. O KNN encontra quase tantos compradores quanto o MLP, mas à custa de 451 alarmes falsos, o que derruba sua precisão para 0,403.

O desbalanceamento influenciou a avaliação? Sim, de três formas. Primeiro, tornou a acurácia enganosa: 84,4% é atingido sem encontrar nenhum comprador, e a acurácia escolheria parametrizações que encontram menos compradores. Segundo, deixou a ROC-AUC otimista: por ela, o KNN parece próximo dos demais (70% do caminho até o modelo perfeito, contra 86% do Random Forest), mas pela PR-AUC a diferença aparece (44% contra 70%), porque a taxa de falsos positivos é diluída pelos milhares de não-compradores. Terceiro, o tratamento do desbalanceamento (sobreamostragem e `class_weight`) desloca os modelos a favor da classe

"comprou": em todos os algoritmos a revocação ficou maior que a precisão, ou seja, os modelos preferem gerar alarmes falsos a perder compradores.

Quais características da base explicam as diferenças? O atributo `PageValues` domina o problema ($\rho = 0{,}63$ com a classe; 56,3% das sessões com `PageValues` > 0 compram, contra 3,9% das demais). As árvores exploram esse tipo de limiar diretamente, não são afetadas pela forte assimetria e pelos outliers e lidam bem com atributos categóricos, o que explica o bom desempenho do Random Forest e do Gradient Boosting. Já o KNN depende de distâncias, e o pré-processamento produziu 55 colunas, das quais 47 vêm do one-hot dos atributos categóricos: duas sessões que diferem apenas no navegador ou no tipo de tráfego ficam tão distantes quanto duas com `PageValues` muito diferentes, e o atributo mais informativo perde peso. A vantagem da distância de Manhattan reforça essa explicação, pois ela não reduz as diferenças pequenas dos atributos numéricos como a euclidiana (que as eleva ao quadrado). O MLP precisou de regularização forte porque a base é pequena para uma rede neural (cerca de 9.800 sessões de treino) e as redes maiores sem regularização decoraram o treino.

Qual modelo recomendamos? Recomendamos o **Random Forest** com 300 árvores e os demais parâmetros no padrão. Ele teve o melhor resultado nas duas medidas escolhidas, é o mais preciso (menos alarmes falsos), foi estável entre os folds e pouco sensível aos parâmetros, não exige normalização e permite inspecionar a importância dos atributos. Numa aplicação real, a escolha final também depende do custo de cada erro: se perder um comprador custar muito mais que um alarme falso (por exemplo, um cupom barato oferecido a quem não compraria), vale reduzir o limiar de decisão do Random Forest, que pelas curvas da Figura 6 atinge a mesma revocação do MLP com precisão semelhante. O Gradient Boosting é uma alternativa praticamente equivalente.

V Conclusões

Principais aprendizados.

- Em problemas desbalanceados, a escolha da medida de avaliação muda as conclusões: a acurácia premiou o modelo que nunca encontra um comprador e teria escolhido parametrizações piores. O F1 e a PR-AUC, focados na classe minoritária, mostraram as diferenças reais.
- O pré-processamento precisa ser pensado para cada algoritmo: as árvores funcionaram bem quase sem transformações, enquanto o KNN e o MLP dependeram de escala, codificação e regularização.
- Ajustar as transformações e a sobreamostragem só nos dados de treino de cada fold, dentro de um pipeline, é essencial para que a avaliação não seja otimista por vazamento de informação.
- Testar vários parâmetros mostrou que alguns algoritmos (KNN e MLP) dependem muito da parametrização, enquanto outros (Random Forest e Gradient Boosting) já funcionam bem com os valores padrão.

Limitações do experimento.

- O teste tem apenas 382 compradores; diferenças pequenas entre algoritmos (como entre Random Forest e Gradient Boosting) estão dentro da variação esperada entre amostras.
- A grade de parâmetros é limitada (72 combinações) e todos os modelos usaram o limiar de decisão padrão de 0,5, sem ajuste ao custo de cada tipo de erro.

- No KNN, o one-hot dos atributos categóricos domina a distância, o que provavelmente subestima o potencial do algoritmo.
- A sobreamostragem aleatória apenas repete sessões, o que pode favorecer o sobreajuste; o par `BounceRates` × `ExitRates` (correlação de 0,91) foi mantido em todos os algoritmos.
- A base cobre um único site e um único ano, então os resultados podem não se generalizar para outras lojas ou períodos.

Possíveis melhorias.

- Ajustar o limiar de decisão de acordo com o custo de cada erro para a loja.
- Melhorar o KNN reduzindo o peso dos atributos categóricos, usando apenas os atributos numéricos ou uma distância própria para dados mistos.
- Testar o SMOTENC no lugar da sobreamostragem aleatória e avaliar a remoção de `BounceRates` ou `ExitRates`.
- Ampliar a busca de parâmetros (busca aleatória ou bayesiana) e usar validação cruzada aninhada para estimar o desempenho com menos variância.
- Testar outras implementações de boosting (XGBoost, LightGBM) e analisar a importância dos atributos para interpretar as decisões do modelo recomendado.

VI Referências

BREIMAN, L. Random forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.

FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. *The Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.

HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.

HUNTER, J. D. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.

KE, G. et al. LightGBM: a highly efficient gradient boosting decision tree. In: *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017.

KINGMA, D. P.; BA, J. Adam: a method for stochastic optimization. In: *International Conference on Learning Representations (ICLR)*, 2015.

LEMAÎTRE, G.; NOGUEIRA, F.; ARIDAS, C. K. Imbalanced-learn: a Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, v. 18, n. 17, p. 1–5, 2017.

MCKINNEY, W. Data structures for statistical computing in Python. In: *Proceedings of the 9th Python in Science Conference*, p. 56–61, 2010.

PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, p. 533–536, 1986.

SAITO, T.; REHMSMEIER, M. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, v. 10, n. 3, e0118432, 2015.

SAKAR, C. O.; POLAT, S. O.; KATIRCIOGLU, M.; KASTRO, Y. Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, v. 31, p. 6893–6908, 2019.

SAKAR, C. O.; KASTRO, Y. Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository, 2018. Disponível em: `https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset`.

Ferramentas: Python 3.13 (`https://www.python.org`), Jupyter (`https://jupyter.org`) e uv (`https://docs.astral.sh/uv`).